Práctica — Reutilizar funciones

Programación Funcional — Lenguaje LISP

Para qué es esta práctica: no es para aprender un tema nuevo. Es para agarrar una sola costumbre: **cuando ya tenés una función que hace algo, la llamás; no la volvés a escribir.**

> **LA REGLA DE LA PRÁCTICA**
>
> Los tres accesores del punto 0 **ya están escritos y te los regalo**. De ahí en adelante:
>
> > **No se puede escribir `car`, `cdr`, `cadr` ni `caddr` sobre un producto.** Si necesitás un dato del producto, llamás a la función que ya existe.
>
> El único `car` y `cdr` permitidos son los de recorrer la lista grande: `(car lista)` para agarrar el producto que estás mirando, y `(cdr lista)` para seguir con el resto.
>
> Y la segunda mitad de la regla, que es la que más cuesta: **tampoco vale volver a definir una función que ya definiste.** Si en el punto 3 escribís `(defun stock-de ...)` de nuevo, perdiste el ejercicio aunque el resultado dé bien.

La verdulería **"La Huerta"** lleva el control de su mercadería en una lista de sublistas. Cada producto es una sublista de **3 elementos**:

```
(PRODUCTO  STOCK  PRECIO)
```

- `PRODUCTO` es un átomo con el nombre.
- `STOCK` son los kilos que quedan.
- `PRECIO` es el precio por kilo.

Punto 0 — Regalado

Estas tres ya están hechas. Copialas tal cual y no las toques más:

```
(defun nombre-de (producto)
    (car producto)
)

(defun stock-de (producto)
    (cadr producto)
)

(defun precio-de (producto)
    (caddr producto)
)
```

Punto 1 — Un producto por vez

1.a) Definir el predicado `hay-poco-p`, que a partir de **un producto** determine si quedan **menos de 5 kilos**. *(Tiene que usar `stock-de`.)*

1.b) Definir `valor-del-producto`, que a partir de **un producto** calcule cuánta plata hay en góndola de ese producto: `STOCK * PRECIO`. *(Tiene que usar `stock-de` y `precio-de`.)*

Punto 2 — Toda la lista

2.a) Definir `contar-sin-stock`, que a partir de la lista completa devuelva **cuántos productos están en cero**. *(Tiene que usar `stock-de`.)*

2.b) Definir el predicado `existe-caro-p`, que a partir de la lista completa y de **un monto recibido como parámetro** determine si **existe al menos un** producto cuyo precio supere ese monto. *(Tiene que usar `precio-de`, y acordate de volver a pasar el monto en cada llamada recursiva.)*

Punto 3 — Acá está el ejercicio de verdad

Estos dos puntos **no usan los accesores**: usan las funciones que escribiste vos en el punto 1.

3.a) Definir `listar-faltantes`, que a partir de la lista completa devuelva una nueva lista con **los nombres** de los productos a los que les queda poco. *(Tiene que usar `hay-poco-p` del punto 1.a y `nombre-de`. No vuelvas a escribir la comparación con el 5.)*

3.b) Definir `valor-total`, que a partir de la lista completa devuelva **la suma de la plata de todos los productos**. *(Tiene que usar `valor-del-producto` del punto 1.b. No vuelvas a escribir la multiplicación.)*

Punto 4 — El informe

4) Definir `generar-cartel`, que a partir de la lista completa devuelva una nueva lista de sublistas `(NOMBRE ETIQUETA)`, donde la etiqueta es:

- `CARO` si el precio es **mayor o igual** a 3000,
- `BARATO` si es menor a 3000.

(Tiene que usar `nombre-de` y `precio-de`.)

ANTES DE ENTREGAR: LA LISTA DE CONTROL

Leé tu código de arriba abajo y marcá:

☐ Después del punto 0, ¿escribí `cadr` o `caddr` en algún lado? Si sí, reemplazalo por la función que corresponde.

☐ ¿Definí dos veces alguna función con el mismo nombre? Buscá `defun` y contá.

☐ En 3.a, ¿aparece el número 5? No tendría que aparecer: eso ya lo sabe `hay-poco-p`.

☐ En 3.b, ¿aparece un `*`? No tendría que aparecer: eso ya lo sabe `valor-del-producto`.

☐ En las funciones con parámetro extra (`existe-caro-p`), ¿lo volví a pasar en **todas** las llamadas recursivas?

☐ ¿Le puse a alguna variable el nombre de una función de LISP (`car`, `list`, `lista`…)?

ENCONTRÁ EL ERROR

Los tres pedazos de abajo están mal. Decí qué tiene cada uno **sin correrlo**.

A)

```
(defun stock-de (car producto)
    (cddr producto))
```

B)

```
(defun contar-caros (lista)
    (cond
        ((null lista) 0)
        ((> (precio-de (car lista)) PRECIO) (+ 1 (contar-caros (cdr lista))))
        (T (contar-caros (cdr lista)))))
```

C)

```
(defun listar-de-precio (lista precio)
    (cond
        ((null lista) NIL)
        ((= (precio-de (car lista)))
            (cons (nombre-de (car lista)) (listar-de-precio (cdr lista) precio)))
        (T (listar-de-precio (cdr lista) precio))))
```

DATOS PARA PROBAR

```
(setq verduleria '((TOMATE 3 2500) (PAPA 20 1200) (LECHUGA 0 3500)
                   (CEBOLLA 2 1800) (ZAPALLO 0 3000) (MORRON 8 4200)))
```